8-11p, DOM input 실행결과

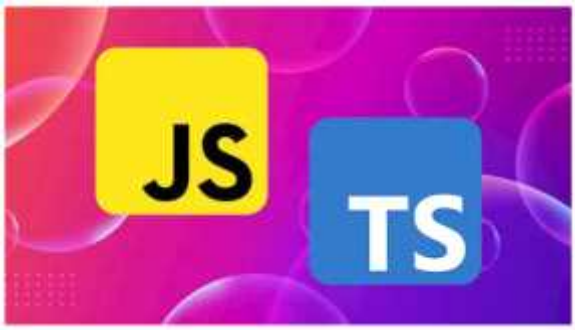


8-11p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8"/>
                <meta       name="viewport"content="width=device-width,
initial-scale=1.0"/>
    <title>DOM Input</title>
    <script src="./8-11p.js"defer></script>
</head>
<body>
    <input type="text"id="userName"value="홍길동"/>
</body>
</html>
```

```
// 1. id가 userName인 입력 요소를 선택
const userName =document.querySelector("#userName");
// 2. 요소가 실제 존재할 때만 value 속성에 접근
if(userName !==null) {
    // value는 입력창에 들어 있는 현재 문자열
    console.log("userName : "+userName.value);
    window.alert("userName : "+userName.value);
}
```

8-13p, DOM Atrribute 실행결과



8-13p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8"/>
                <meta       name="viewport"content="width=device-width,
initial-scale=1.0"/>
    <title>DOM Attribute</title>
    <script src="./8-13p.js"defer></script>
</head>
<body>
        <img id="logo"src="image1.jpg"alt="기존 이미지"width="240"/>
</body>
</html>
```

```
// 1. id가 logo인 img 요소를 선택
const logo =document.querySelector("#logo");
if(logo !==null) {
    // 2. img의 src/alt HTML 속성을 변경
    logo.setAttribute("src", "image2.jpg");
    logo.setAttribute("alt", "변경된 이미지");
    // 3. 변경된 src 값을 콘솔에 출력
    console.log(logo.getAttribute("src"));
}
```

8-14p, DOM style 실행결과



8-14p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8"/>
                <meta       name="viewport"content="width=device-width,
initial-scale=1.0"/>
    <title>DOM Style</title>
    <script src="./8-14p.js"defer></script>
</head>
<body>
    <h1 id="title">Frontend Programming</h1>
</body>
</html>
```

```
// 1. 제목 요소를 선택
const title =document.querySelector("#title");
if(title !==null) {
    // 2. JavaScript의 style 프로퍼티로 CSS를 설정
    title.style.color ="blue";           // 글자색
         title.style.backgroundColor    ="yellow";    //    배경색    (CSS
background-color)
    title.style.fontSize ="40px";        // 글자 크기 (CSS font-size)
}
```

8-15p, DOM ClassList 실행결과



8-15p html, js

```html
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8"/>
                <meta       name="viewport"content="width=device-width,
initial-scale=1.0"/>
    <title>DOM ClassList</title>
    <style>
        .highlight{
            color: blue;
            background-color: yellow;
            font-size: 40px;
}
    </style>
    <script src="./8-15p.js"defer></script>
</head>
<body>
    <h1 id="title">Frontend Programming</h1>
</body>
</html>
```

```javascript
// 1. 제목 요소를 선택
const title =document.querySelector("#title");
if(title !==null) {
    // 2. CSS에 미리 작성한 highlight 클래스를 적용
    title.classList.add("highlight");
}
```

8-18p, DOM create 실행결과

새로운 DOM 요소입니다.

8-18p html, js

```html
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8"/>
                <meta       name="viewport"content="width=device-width,
initial-scale=1.0"/>
```

```
    <title>DOM Create</title>
    <script src="./8-18p.js"defer></script>
</head>
<body>
    <div id="container"></div>
</body>
</html>
```

```
// 1. 새 요소를 넣을 부모 div를 선택
const container =document.querySelector("#container");
// 2. 아직 HTML 문서에 연결되지 않은 p 요소를 생성
const message =document.createElement("p");
// 3. 새 p 요소의 텍스트를 지정
message.textContent ="새로운 DOM 요소입니다.";
// 4. 부모가 존재하면 새 요소를 부모의 자식으로 추가
if(container !==null) {
    container.append(message);
}
```

8-19p, DOM QuerySelectorAll 실행결과

- HTML
- CSS
- JavaScript
- TypeScript

8-19p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8"/>
                <meta       name="viewport"content="width=device-width,
initial-scale=1.0"/>
    <title>DOM QuerySelectorAll</title>
    <script src="./8-19p.js"defer></script>
</head>
<body>
    <ul>
```

```
        <li class="item">HTML</li>
        <li class="item">CSS</li>
        <li class="item">JavaScript</li>
        <li class="item">TypeScript</li>
    </ul>
</body>
</html>
```

```
// 1. .item에 일치하는 모든 요소를 NodeList로 가져오기
const items =document.querySelectorAll(".item");
// 2. forEach()로 각 li의 텍스트를 순서대로 출력
items.forEach((item) =>{
    console.log(item.textContent);
});
```

8-20p, DOM Remove 실행결과

기본 p 태그

8-20p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8"/>
                <meta       name="viewport"content="width=device-width,
initial-scale=1.0"/>
    <title>DOM Remove</title>
    <script src="./8-20p.js"defer></script>
</head>
<body>
    <p id="message">삭제될 메시지입니다.</p>
</body>
</html>
```

```
// 1. 삭제할 문단을 선택
const message =document.querySelector("#message");
// 2. DOM에서 해당 요소를 제거
if(message !==null) {
    message.remove();
```

```
}
```

9-4p clickEvent 실행결과

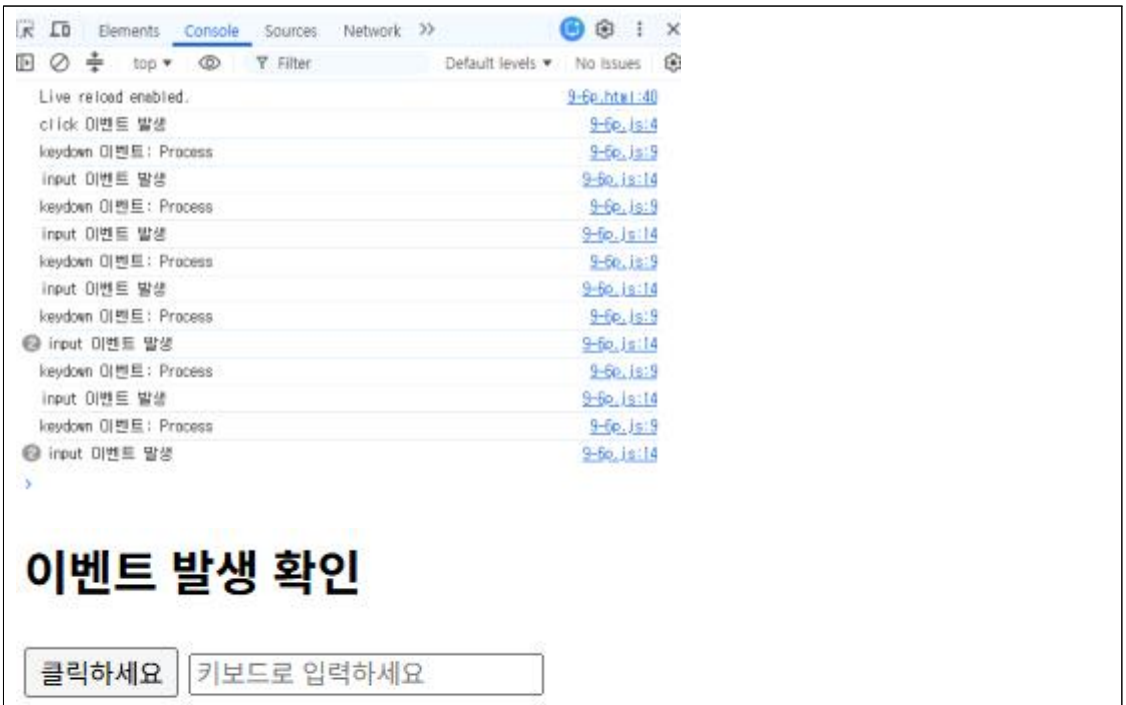


9-4p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8"/>
<title>클릭 이벤트 실습</title>
</head>
<body>
<h1>클릭 이벤트 실습</h1>
<button id="btn">클릭</button>
<p id="message">대기 중</p>
<script src="9-4p.js"defer></script>
</body>
</html>
```

```
const btn =document.querySelector("#btn");
const message =document.querySelector("#message");
if(btn &&message) {
    btn.addEventListener("click", () =>{
    message.textContent ="버튼을 클릭했습니다!";
});
}
```

9-6p 이벤트 종류 실행결과



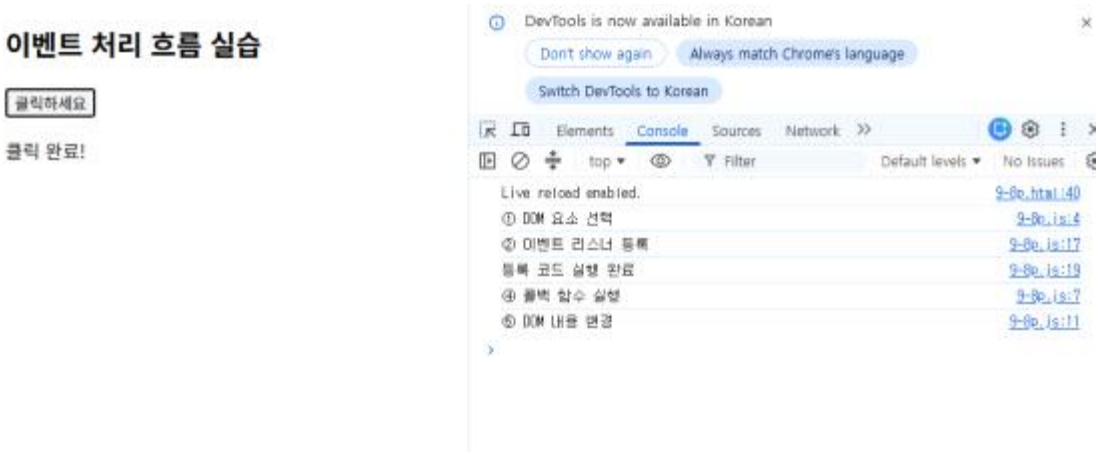
9-6p html, js

```html
<!DOCTYPE html>
<html lang="ko">
    <head>
        <meta charset="UTF-8"/>
        <title>이벤트 종류 실습</title>
        <script src="9-6p.js"defer></script>
    </head>
    <body>
        <h2>이벤트 발생 확인</h2>
        <button id="myButton">클릭하세요</button>
        <input id="myInput"type="text"placeholder="키보드로 입력하세요"/>
    </body>
</html>
```

```javascript
// 버튼 클릭 이벤트
const button =document.querySelector("#myButton");
button?.addEventListener("click", () =>{
console.log("click 이벤트 발생");
});
// 키보드 입력 이벤트
const input =document.querySelector("#myInput");
```

```
    input?.addEventListener("keydown", (event) =>{
    console.log("keydown 이벤트:", event.key);
});
// 입력값 변경 이벤트
input?.addEventListener("input", (event) =>{
console.log("input 이벤트 발생");
});
```

9-8p 이벤트 흐름 순서 실행결과



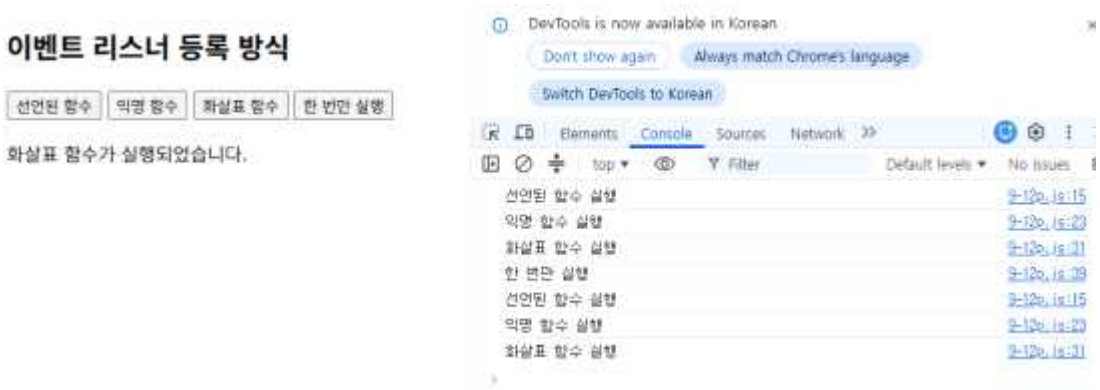
9-8p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8"/>
<title>이벤트 처리 흐름</title>
<script src="9-8p.js"defer></script>
</head>
<body>
    <h2>이벤트 처리 흐름 실습</h2>
    <button id="btn">클릭하세요</button>
    <p id="message">버튼 클릭을 기다리는중...</p>
</body>
</html>
```

```
// ① DOM 요소 선택
const button =document.querySelector("#btn");
const message =document.querySelector("#message");
console.log("① DOM 요소 선택");
```

```
// ④ 이벤트 발생 시 실행할 콜백 함수
function handleClick() {
console.log("④ 콜백 함수 실행");
// ⑤ DOM 변경
 if(message !==null) {
message.textContent ="클릭 완료!";
console.log("⑤ DOM 내용 변경");
}
}
// ② 이벤트 리스너 등록
if(button !==null) {
button.addEventListener("click", handleClick);
console.log("② 이벤트 리스너 등록");
}
console.log("등록 코드 실행 완료");
// ③ 사용자가 실제로 버튼을 클릭할 때 브라우저에서 click 이벤트가 발생하고 등
록한 handleClick 함수가 호출됨
```

9-12p 실행결과



9-12p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8"/>
  <title>addEventListener 실습</title>

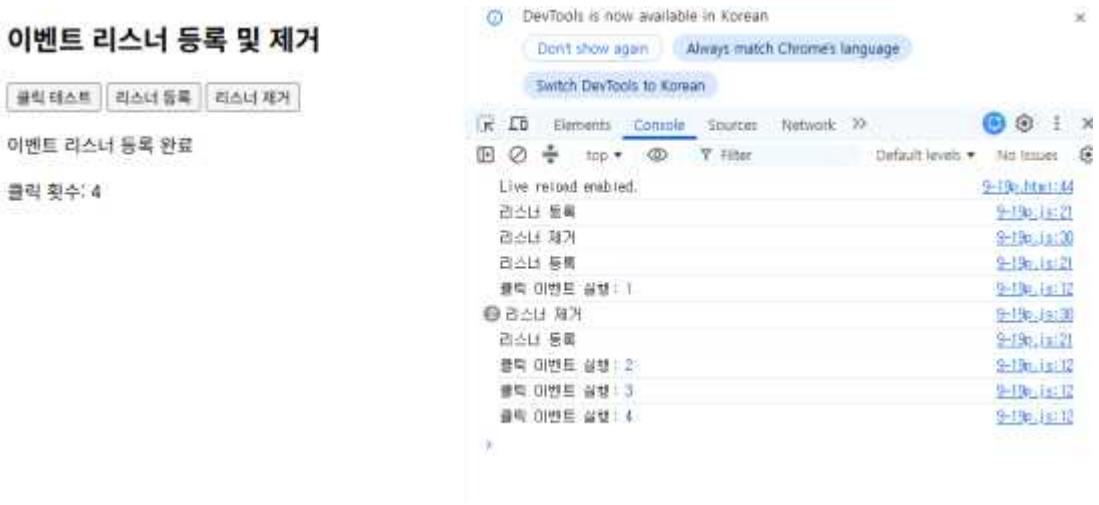
  <script src="./9-12p.js"defer></script>
</head>
<body>
```

```
  <h2>이벤트 리스너 등록 방식</h2>
  <button id="btn1">선언된 함수</button>
  <button id="btn2">익명 함수</button>
  <button id="btn3">화살표 함수</button>
  <button id="btn4">한 번만 실행</button>
  <p id="message">버튼을 클릭하세요.</p>
</body>
</html>
```

```
// 1. DOM 요소 선택
const btn1 =document.querySelector("#btn1");
const btn2 =document.querySelector("#btn2");
const btn3 =document.querySelector("#btn3");
const btn4 =document.querySelector("#btn4");
const message =document.querySelector("#message");
// 메시지 변경을 재사용하는 함수
function showMessage(text) {
  if(message !==null) message.textContent =text;
}
// 2. 선언된 함수 등록
function handleClick() {
  console.log("선언된 함수 실행");
  showMessage("선언된 함수가 실행되었습니다.");
}
if(btn1 !==null) btn1.addEventListener("click", handleClick);
// 3. 익명 함수 등록
if(btn2 !==null) {
  btn2.addEventListener("click", function() {
    console.log("익명 함수 실행");
    showMessage("익명 함수가 실행되었습니다.");
});
}
// 4. 화살표 함수 등록
if(btn3 !==null) {
  btn3.addEventListener("click", () =>{
    console.log("화살표 함수 실행");
    showMessage("화살표 함수가 실행되었습니다.");
});
}
```

```
// 5. once: true → 한 번만 실행한 뒤 자동으로 리스너 제거
if(btn4 !==null) {
  btn4.addEventListener("click", () =>{
    console.log("한 번만 실행");
    showMessage("이 버튼의 리스너는 한 번만 실행됩니다.");
}, { once:true});
}
```

9-19p 실행결과



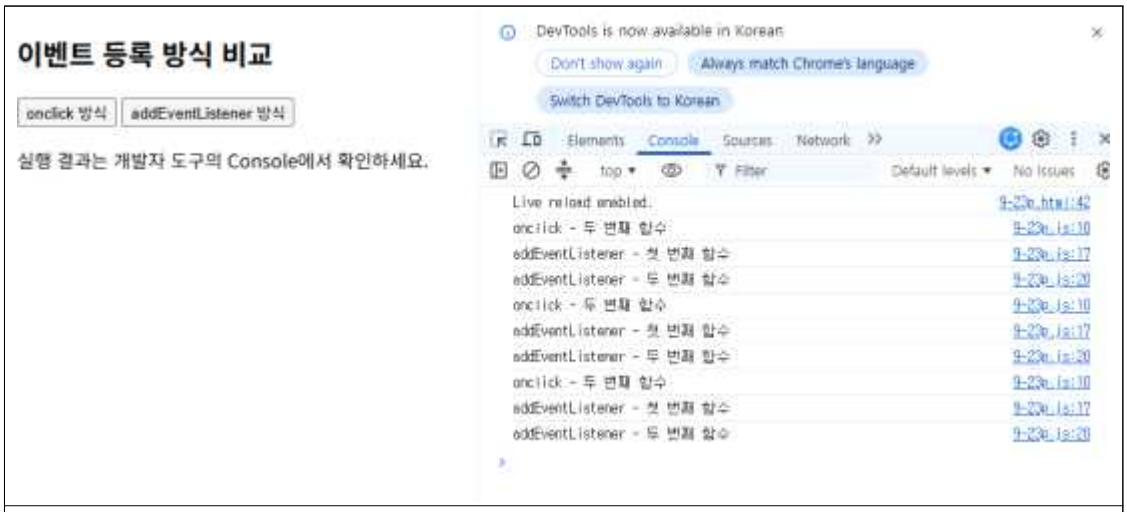
9-19 html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8"/>
  <title>이벤트 리스너 제거</title>

  <script src="./9-19p.js"defer></script>
</head>
<body>
  <h2>이벤트 리스너 등록 및 제거</h2>
  <button id="clickBtn">클릭 테스트</button>
  <button id="addBtn">리스너 등록</button>
  <button id="removeBtn">리스너 제거</button>
  <p id="message">등록 버튼을 눌러주세요.</p>
  <p id="count">클릭 횟수: 0</p>
</body>
</html>
```

```
// 1. DOM 요소 선택
const clickBtn =document.querySelector("#clickBtn");
const addBtn =document.querySelector("#addBtn");
const removeBtn =document.querySelector("#removeBtn");
const message =document.querySelector("#message");
const countText =document.querySelector("#count");
let count =0;
// 2. 클릭 이벤트에 사용할 함수 (제거 시 같은 함수 참조 필요)
function handleClick() {
  count++;
  console.log("클릭 이벤트 실행:", count);
  if(countText !==null) countText.textContent =`클릭 횟수: ${count}`;
}
// 3. '리스너 등록' 버튼 클릭 시 클릭 테스트 버튼에 등록
if(addBtn !==null &&clickBtn !==null) {
  addBtn.addEventListener("click", () =>{
    clickBtn.addEventListener("click", handleClick);
    if(message !==null) message.textContent ="이벤트 리스너 등록 완료";
    console.log("리스너 등록");
});
}
// 4. '리스너 제거' 버튼 클릭 시 동일한 콜백을 제거
if(removeBtn !==null &&clickBtn !==null) {
  removeBtn.addEventListener("click", () =>{
    clickBtn.removeEventListener("click", handleClick);
    if(message !==null) message.textContent ="이벤트 리스너 제거 완료";
    console.log("리스너 제거");
});
}
```

9-23p 실행결과



9-23p html, js

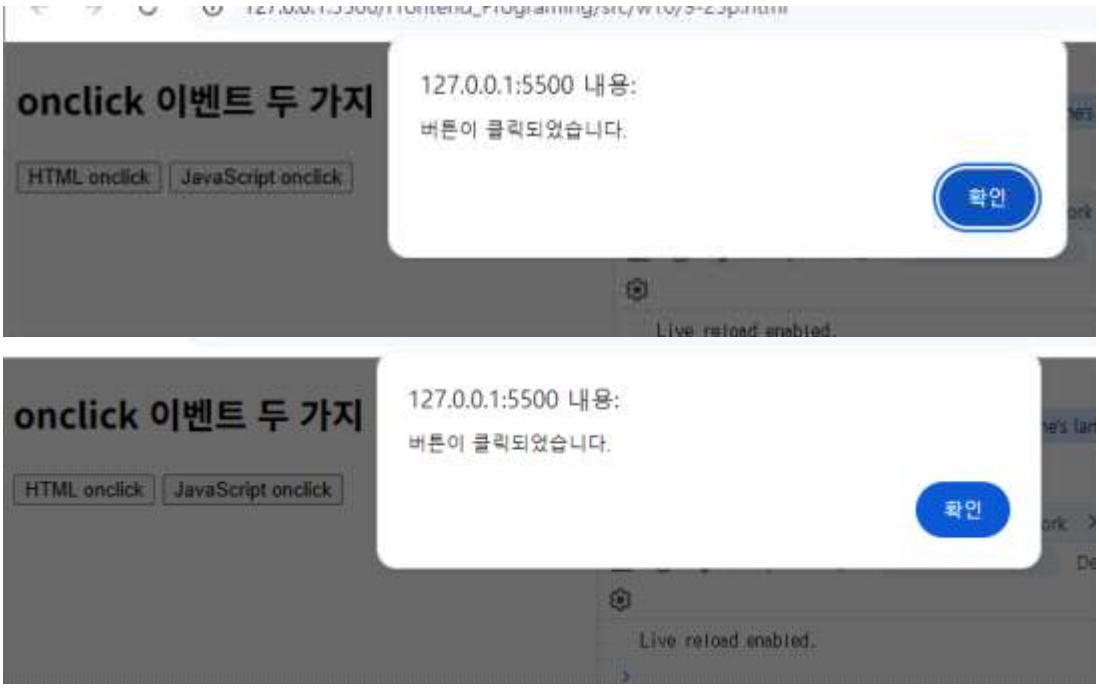
```html
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8"/>
  <title>onclick과 addEventListener 비교</title>

  <script src="./9-23p.js"defer></script>
</head>
<body>
  <h2>이벤트 등록 방식 비교</h2>
  <button id="btn1">onclick 방식</button>
  <button id="btn2">addEventListener 방식</button>
  <p>실행 결과는 개발자 도구의 Console에서 확인하세요.</p>
</body>
</html>
```

```javascript
const btn1 =document.querySelector("#btn1");
const btn2 =document.querySelector("#btn2");
// 1. onclick 프로퍼티: 나중에 할당한 함수가 기존 함수를 대체
if(btn1 !==null) {
  btn1.onclick =function() {
    console.log("onclick - 첫 번째 함수");
};
  btn1.onclick =function() {
    console.log("onclick - 두 번째 함수");
};
```

```
}
// 2. addEventListener: 같은 이벤트에도 여러 함수 등록 가능
if(btn2 !==null) {
  btn2.addEventListener("click", function() {
    console.log("addEventListener - 첫 번째 함수");
});
  btn2.addEventListener("click", function() {
    console.log("addEventListener - 두 번째 함수");
});
}
```

9-25p 실행결과



9-25p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8"/>
  <title>onclick 이벤트 실습</title>

  <script src="./9-25p.js"defer></script>
</head>
```

```
<body>
  <h2>onclick 이벤트 두 가지 방식</h2>
  <!-- HTML 속성에서 함수 호출 -->
  <button onclick="myFunction()">HTML onclick</button>
  <!-- JavaScript에서 onclick 프로퍼티 등록 -->
  <button id="btnHello">JavaScript onclick</button>
</body>
</html>
```

```
// PDF 25쪽의 두 onclick 방식 예제를 한 페이지에서 실행하도록 통합
// 1. HTML의 onclick="myFunction()"에서 호출되는 함수
function myFunction() {
  alert("버튼이 클릭되었습니다.");
}
// 2. JavaScript에서 onclick 프로퍼티에 함수 할당
const btnHello =document.getElementById("btnHello");
if(btnHello !==null) {
  btnHello.onclick =function() {
    alert("버튼이 클릭되었습니다.");
};
}
```

9-28p 실행결과



9-28p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8"/>
  <title>이벤트 객체 실습</title>
```

```
  <script src="./9-28p.js"defer></script>
</head>
<body>
  <h2>이벤트 객체 확인</h2>
  <button id="btn"><span>버튼 안의 텍스트 클릭</span></button>
  <p id="result">클릭 이벤트 발생 대기 중...</p>
</body>
</html>
```

```
// 1. DOM 요소 선택
const button =document.querySelector("#btn");
const result =document.querySelector("#result");
// 2. 브라우저가 넘겨주는 이벤트 객체 활용
function handleClick(event) {
  console.log("이벤트 종류:", event.type);
  console.log("이벤트 발생 대상:", event.target);
  console.log("리스너 등록 요소:", event.currentTarget);
  console.log("X 좌표:", event.clientX);
  console.log("Y 좌표:", event.clientY);
  // span을 클릭했다면 target은 SPAN, currentTarget은 BUTTON
  const target =event.target;
  const currentTarget =event.currentTarget;
   if(result  !==null  &&target  instanceof  Element  &&currentTarget
instanceof Element) {
          result.textContent    =`이벤트:    ${event.type},    target:
${target.tagName}, currentTarget: ${currentTarget.tagName}`;
}
}
// 3. 이벤트 리스너 등록
if(button !==null) button.addEventListener("click", handleClick);
```

9-31p 실행결과



9-31p html, js

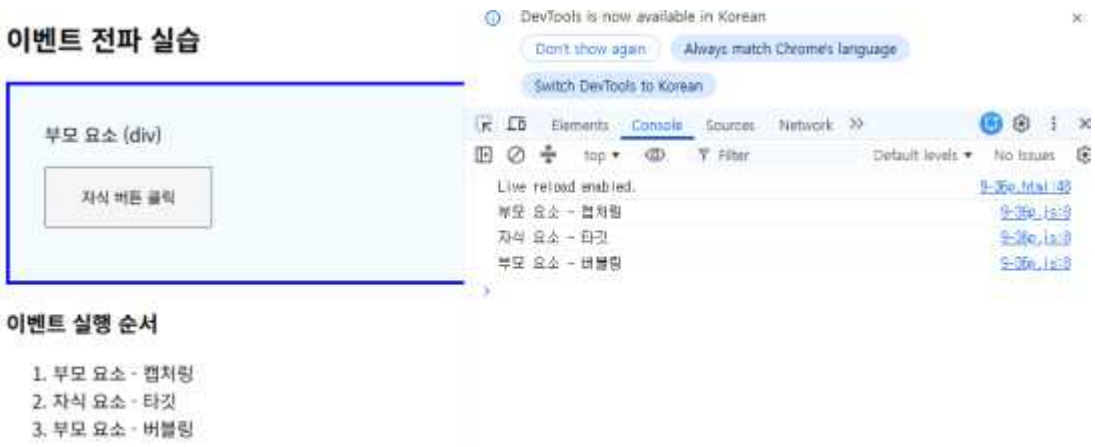
```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8"/>
  <title>키보드와 마우스 이벤트</title>

  <script src="./9-31p.js"defer></script>
</head>
<body>
  <h2>키보드 이벤트</h2>
  <input id="keyInput"type="text"placeholder="키보드를 입력하세요"/>
  <p id="keyResult">입력 대기 중...</p>
  <hr />
  <h2>마우스 이벤트</h2>
   <div  id="mouseArea"style="width:400px;height:180px;border:2px  solid
blue;background-color:#eef5ff;padding:10px;">
이 영역에서 마우스를 움직이거나 클릭하세요.
  </div>
  <p id="mouseResult">마우스 이벤트 대기 중...</p>
</body>
</html>
```

```
// 1. DOM 요소 선택
const keyInput =document.querySelector("#keyInput");
const keyResult =document.querySelector("#keyResult");
const mouseArea =document.querySelector("#mouseArea");
const mouseResult =document.querySelector("#mouseResult");
// 2. 키보드 이벤트
if(keyInput !==null &&keyResult !==null) {
  keyInput.addEventListener("keydown", (event) =>{
```

```
      keyResult.textContent =`keydown | key: ${event.key}, code:
${event.code}, repeat: ${event.repeat}`;
    console.log("keydown:", event.key);
});
  keyInput.addEventListener("keyup", (event) =>{
    console.log("keyup:", event.key);
});
}
// 3. 마우스 이벤트
if(mouseArea !==null &&mouseResult !==null) {
  mouseArea.addEventListener("mousemove", (event) =>{
        mouseResult.textContent =`마우스 좌표: (${event.clientX},
${event.clientY})`;
});
  mouseArea.addEventListener("mousedown", (event) =>{
    console.log("mousedown:", event.button);
});
  mouseArea.addEventListener("mouseup", (event) =>{
    console.log("mouseup:", event.button);
});
  mouseArea.addEventListener("click", () =>{
    console.log("click 이벤트 발생");
});
}
```

9-36p 실행결과



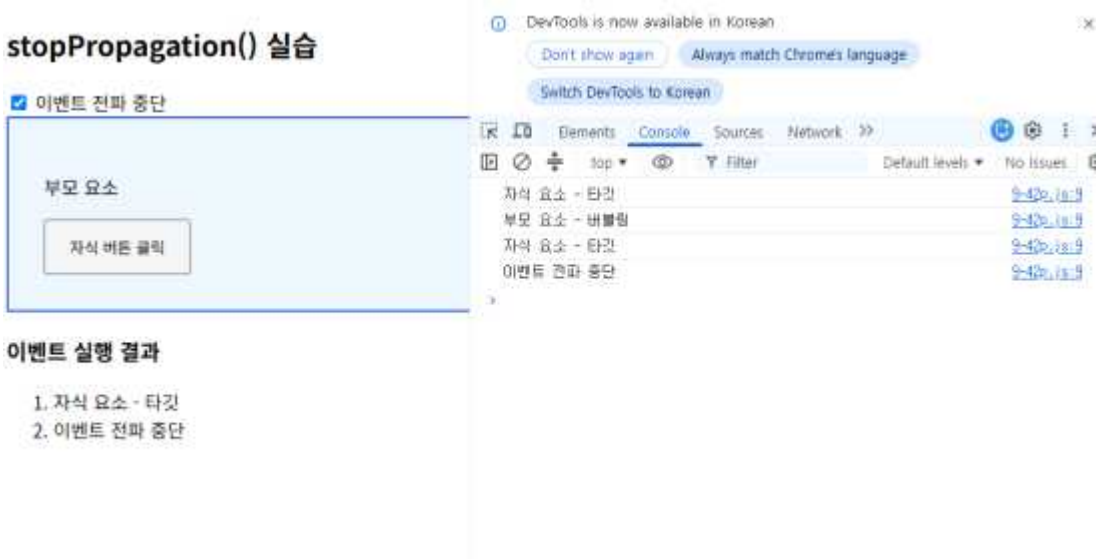
9-36p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8"/>
  <title>이벤트 전파 실습</title>
  <style>
#parentwidth:  350px;  padding:  30px;  border:  3px  solid  blue;
background-color: #eef4ff; }
#childpadding: 15px 30px; cursor: pointer; }
</style>
  <script src="./9-36p.js"defer></script>
</head>
<body>
  <h2>이벤트 전파 실습</h2>
  <div id="parent">
부모 요소 (div)
    <p><button id="child">자식 버튼 클릭</button></p>
  </div>
  <h3>이벤트 실행 순서</h3>
  <ol id="result"></ol>
</body>
</html>
```

```
// 1. DOM 요소 선택
const parent =document.querySelector("#parent");
const child =document.querySelector("#child");
const result =document.querySelector("#result");
// 2. 결과 출력 함수
function showResult(message) {
  console.log(message);
  if(result !==null) {
    const li =document.createElement("li");
    li.textContent =message;
    result.appendChild(li);
}
}
// 3. 캡처링: 부모 → 자식으로 내려가며 먼저 실행
if(parent !==null) {
  parent.addEventListener("click", () =>{
```

```
    if(result !==null) result.replaceChildren();
    showResult("부모 요소 - 캡처링");
}, { capture:true});
}
// 4. 타깃: 자식 버튼 클릭 시 실행
if(child !==null) {
  child.addEventListener("click", () =>{
    showResult("자식 요소 - 타깃");
});
}
// 5. 버블링: 자식 → 부모로 올라오며 실행
if(parent !==null) {
  parent.addEventListener("click", () =>{
    showResult("부모 요소 - 버블링");
}, { capture:false});
}
```

9-42p 실행결과



9-42p html, js

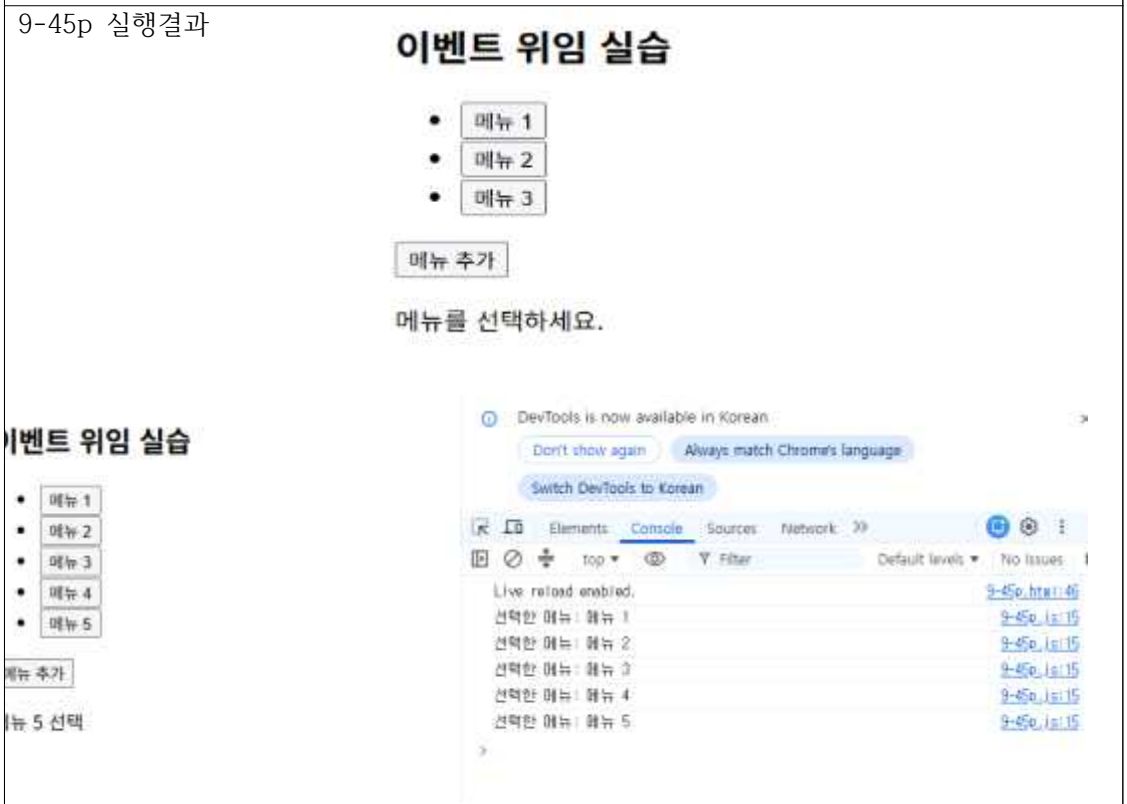
```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8"/>
  <title>이벤트 전파 제어</title>
  <style>
#parentwidth: 350px; padding: 30px; background-color: #e7f0ff; border:
```

```
2px solid royalblue; }
#childpadding: 12px 20px; cursor: pointer; }
</style>
  <script src="./9-42p.js"defer></script>
</head>
<body>
  <h2>stopPropagation() 실습</h2>
  <label><input type="checkbox"id="stopCheck"/>이벤트 전파 중단</label>
  <div id="parent">
    <p>부모 요소</p>
    <button id="child">자식 버튼 클릭</button>
  </div>
  <h3>이벤트 실행 결과</h3>
  <ol id="result"></ol>
</body>
</html>
```

```
// 1. DOM 요소 선택
const parent =document.querySelector("#parent");
const child =document.querySelector("#child");
const stopCheck =document.querySelector("#stopCheck");
const result =document.querySelector("#result");
// 2. 결과 출력 함수
function showResult(message) {
  console.log(message);
  if(result !==null) {
    const li =document.createElement("li");
    li.textContent =message;
    result.appendChild(li);
}
}
// 3. 부모의 버블링 리스너
if(parent !==null) {
  parent.addEventListener("click", () =>{
    showResult("부모 요소 - 버블링");
});
}
// 4. 자식의 리스너
if(child !==null) {
```

```
  child.addEventListener("click", (event) =>{
    if(result !==null) result.replaceChildren();
    showResult("자식 요소 - 타깃");
    if(stopCheck !==null &&stopCheck.checked) {
      // 체크 시 이벤트가 부모까지 올라가지 않도록 차단
      event.stopPropagation();
      showResult("이벤트 전파 중단");
}
});
}
```

9-45p 실행결과



9-45p html, js

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8"/>
  <title>이벤트 위임 실습</title>

  <script src="./9-45p.js"defer></script>
</head>
```

```
<body>
  <h2>이벤트 위임 실습</h2>
  <ul id="menu">
    <li><button type="button">메뉴 1</button></li>
    <li><button type="button">메뉴 2</button></li>
    <li><button type="button">메뉴 3</button></li>
  </ul>
  <button id="addBtn"type="button">메뉴 추가</button>
  <p id="result">메뉴를 선택하세요.</p>
</body>
</html>
```

```
// 1. DOM 요소 선택
const menu =document.querySelector("#menu");
const addBtn =document.querySelector("#addBtn");
const result =document.querySelector("#result");
let menuCount =3;
// 2. 이벤트 위임: 각각의 버튼 대신 부모 ul에 리스너 하나만 등록
if(menu !==null) {
  menu.addEventListener("click", (event) =>{
    // 이벤트 발생 대상이 HTML 요소인지 확인
    if(!(event.target instanceof Element)) return;
    // 클릭한 요소나 상위 요소 중 가장 가까운 button 탐색
    const button =event.target.closest("button");
    if(button ===null ||!menu.contains(button)) return;
    console.log("선택한 메뉴:", button.textContent);
    if(result !==null) result.textContent =`${button.textContent}선택`;
});
}
// 3. 동적으로 새 버튼 생성 (새 버튼에도 위임 리스너가 작동)
if(addBtn !==null &&menu !==null) {
  addBtn.addEventListener("click", () =>{
    menuCount++;
    const li =document.createElement("li");
    const button =document.createElement("button");
    button.type ="button";
    button.textContent =`메뉴 ${menuCount}`;
    li.appendChild(button);
    menu.appendChild(li);
```

```
});
}
```